

NetProbe

UDP Tabanlı Güvenilir Dosya Aktarımı,
Trafik İzleme ve Ağ Performans Analiz Platformu

Bursa Teknik Üniversitesi
Bilgisayar Mühendisliği Bölümü
Bilgisayar Ağları Dersi — Dönem Projesi

Teknik Rapor

İçindekiler

1. Giriş ve Proje Özeti
 2. NetProbe Nedir? Ne İşe Yarar?
 3. Sistem Mimarisi
 4. Çalışma Mantığı ve Protokol
 5. Kurulum ve Kullanım Kılavuzu
 6. Güvenilirlik Mekanizmaları
 7. Loglama ve Performans Metrikleri
 8. Deneysel Çalışma ve Sonuçlar
 9. Sonuç ve Değerlendirme
- Kaynaklar ve Ekler

1. Giriş ve Proje Özeti

Bu rapor, Bilgisayar Ağları dersi dönem projesi kapsamında geliştirilen NetProbe sisteminin tasarımını, çalışma prensibini, kullanımını ve deneysel performans sonuçlarını sunmaktadır. NetProbe, UDP protokolü üzerinde uygulama katmanında güvenilir dosya aktarımı sağlayan; trafik olaylarını kaydeden ve ağ performans metriklerini ölçen entegre bir platformdur.

Projenin temel motivasyonu, TCP'nin sunduğu güvenilirlik mekanizmalarının (sıralama, onay, yeniden gönderim) UDP üzerinde sıfırdan nasıl tasarlanabileceğini uygulamalı olarak göstermektir.

2. NetProbe Nedir? Ne İşe Yarar?

2.1 Tanım

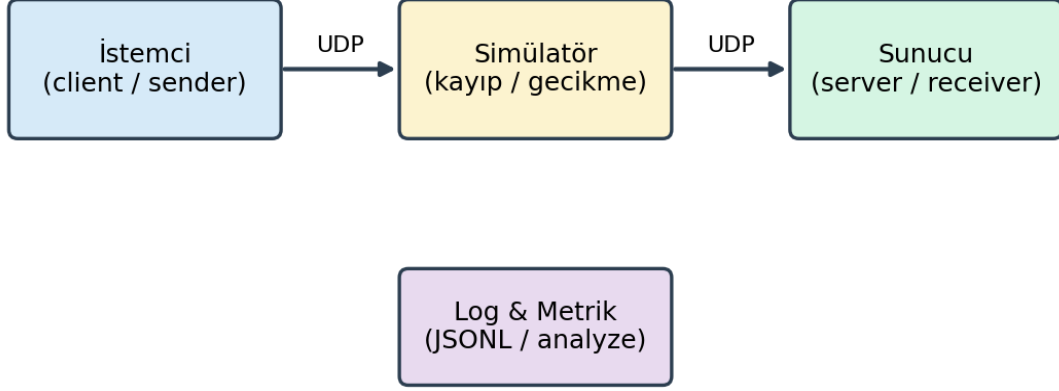
UDP bağlantısız ve güvenilir bir taşıma protokolüdür: paket sırası garanti edilmez, kayıp paketler otomatik yeniden iletilmez. NetProbe bu eksiklikleri uygulama katmanında sequence number, ACK, timeout, retransmission, CRC32 checksum ve SHA256 hash ile giderir.

2.2 Kullanım Alanları

- UDP ve TCP arasındaki farkları uygulamalı öğrenmek
- Basit bir uygulama katmanı protokolü tasarlamak
- Ağ trafiğini loglayarak protokol davranışını gözlemlemek
- Paket boyutu, timeout ve kayıp oranının performansa etkisini ölçmek

3. Sistem Mimarisi

Sistem dört ana bileşenden oluşur: İstemci (dosya gönderimi), Simülatör (opsiyonel kayıp/gecikme), Sunucu (dosya alımı ve birleştirme) ve Analiz (metrik ve grafik üretimi).

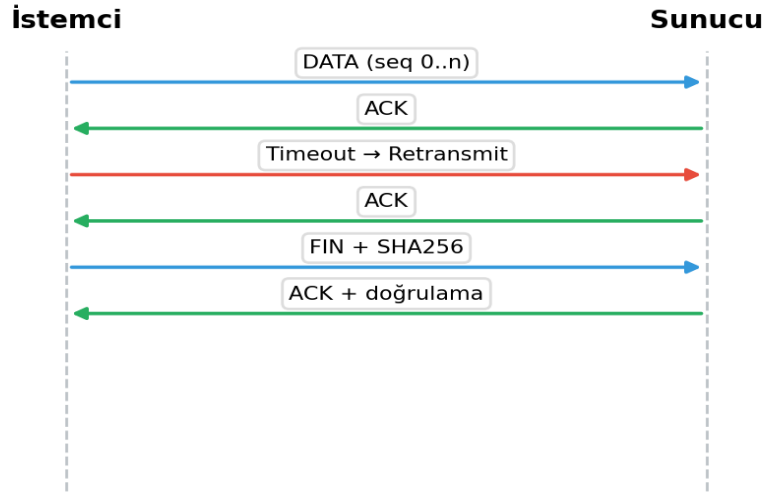


Şekil 1. NetProbe sistem mimarisi ve bileşenler arası veri akışı

Bileşen	Dosya	Görev
İstemci	client.py, sender.py	Sliding window ile dosya gönderimi
Sunucu	server.py, receiver.py	Selective repeat ile alım ve birleştirme
Simülatör	simulator.py	UDP proxy; yapay kayıp ve gecikme
Analiz	logger.py, metrics.py, analyze.py	Loglama ve performans raporu

4. Çalışma Mantığı ve Protokol

4.1 Aktarım Akışı



Şekil 2. Tipik paket alışverişi (DATA, ACK, retransmit, FIN)

- Adım 1: Client dosyayı chunk'lara böler ve her parçaya sequence number atar.
- Adım 2: Sliding window (varsayılan 8) ile birden fazla paket paralel gönderilir.
- Adım 3: Server her geçerli DATA için ACK döner; duplicate paketler tekrar yazılmaz.
- Adım 4: Timeout durumunda client paketi yeniden gönderir (en fazla 5 deneme).
- Adım 5: Tüm parçalar alındığında client FIN paketi ile SHA256 hash gönderir.
- Adım 6: Server dosyayı birleştirir ve hash değerini doğrular.

4.2 Paket Türleri

Tür	Kod	Açıklama
DATA	1	Dosya parçası; seq, total_pkts, payload, CRC32
ACK	2	Onay; ack_num
FIN	3	Aktarım sonu; SHA256 hash + dosya adı
ERR	4	Hata bildirimi

5. Kurulum ve Kullanım Kılavuzu

5.1 Kurulum

```
cd netprobe
pip install -r requirements.txt
export PYTHONPATH=src
```

5.2 Kayıpsız Aktarım (localhost)

Terminal 1: python src/server.py --port 9001 --out-dir received/

Terminal 2: python src/client.py --port 9001 --file dosya.bin

5.3 Kayıplı Ortam (simülatör ile)

Localhost gerçek paket kaybı üretmez. Bu nedenle deneylerde UDP proxy simülatörü kullanılır: istemci porta 9000'e bağlanır, simülatör paketleri %X oranında düşürerek sunucuya (9001) iletir.

Terminal 1: python src/simulator.py --listen 9000 --forward 9001 --loss-rate 0.05

Terminal 2: python src/server.py --port 9001

Terminal 3: python src/client.py --port 9000 --file dosya.bin

Parametre	Varsayılan	Açıklama
CHUNK_SIZE	1024 B	Dosya parça boyutu
WINDOW_SIZE	8	Sliding window genişliği
TIMEOUT_MS	1000 ms	ACK bekleme süresi
MAX_RETRIES	5	Paket başına max yeniden gönderim

6. Güvenilirlik Mekanizmaları

Mekanizma	Uygulama	Föy Gereksinimi
Sequence number	Her paket numaralandırılır	Zorunlu
ACK	Her başarılı DATA onaylanır	Zorunlu
Timeout	select() tabanlı zaman aşımı	Zorunlu
Retransmit	Max 5 deneme, sonra hata	Zorunlu
Duplicate	ACK tekrar, diske yazılmaz	Zorunlu
SHA256	FIN ile hash doğrulama	Zorunlu
Sliding window	Paralel 8 paket	Bonus

7. Loglama ve Performans Metrikleri

Tüm olaylar JSONL formatında kaydedilir: send, ack_received, timeout, retransmit, transfer_complete, error. Transfer sonunda metrics.json dosyası üretilir.

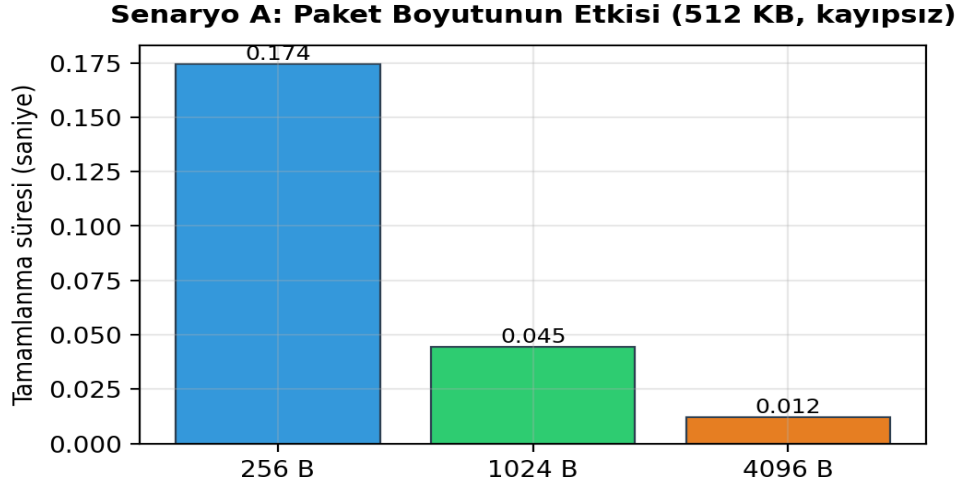
Metrik	Tanım
Throughput	$\text{file_bytes} \times 8 / \text{completion_time}$ (bps)
Goodput	Bu projede throughput ile aynı tanım
Retransmission rate	retransmit sayısı / gönderilen paket
Packet loss rate	timeout sayısı / gönderilen paket
avg_rtt_ms	Ortalama ACK gecikmesi (ms)

8. Deneysel Çalışma ve Sonuçlar

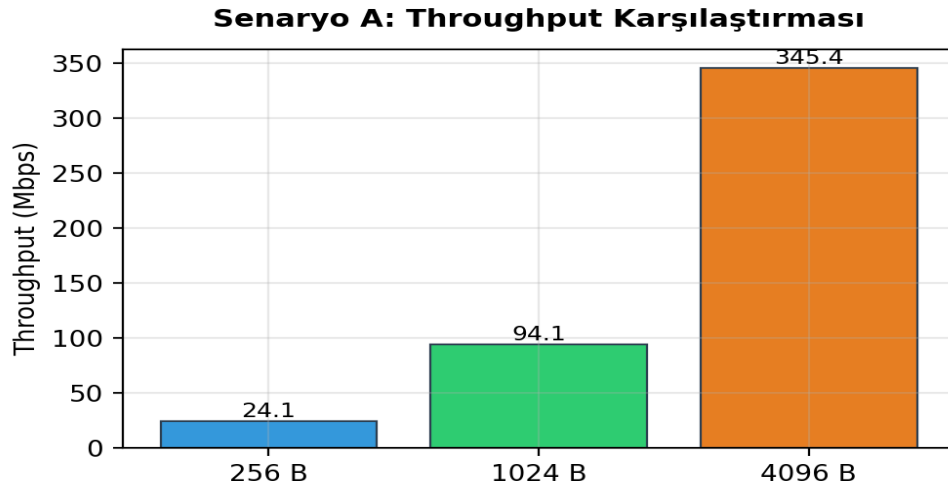
8.1 Deney Ortamı

İşletim sistemi: macOS / Linux; Python 3.11; localhost UDP. Kayıp deneyleri simulator.py proxy üzerinden yapılmıştır. Her senaryo 3 tekrarlı çalıştırılmış, tablolarda ortalama değerler verilmiştir.

8.2 Senaryo A — Paket Boyutu (512 KB, kayıpsız)



Şekil 3. Paket boyutunun tamamlanma süresine etkisi

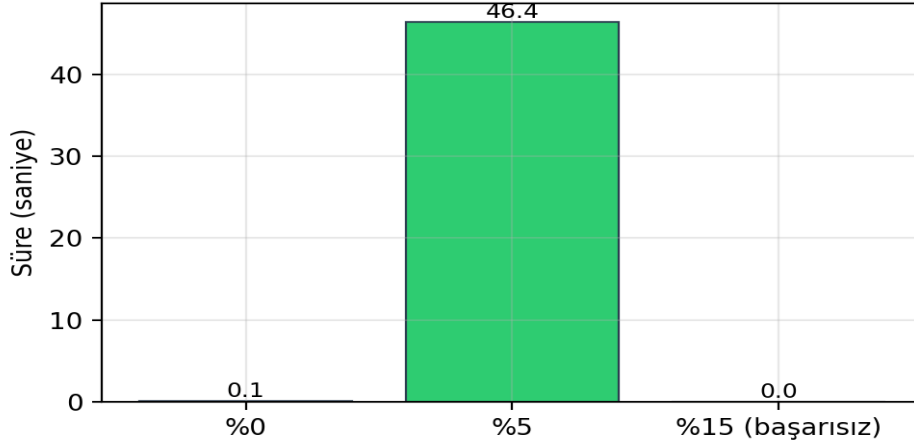


Şekil 4. Paket boyutunun throughput değerine etkisi

Yorum: Ölçülen tamamlanma süreleri — 256 B: 0.174 s, 1024 B: 0.045 s, 4096 B: 0.012 s. Küçük chunk daha fazla paket ürettiğinden header overhead artar ve süre uzar. Büyük chunk paket sayısını azaltarak verimi yükseltir.

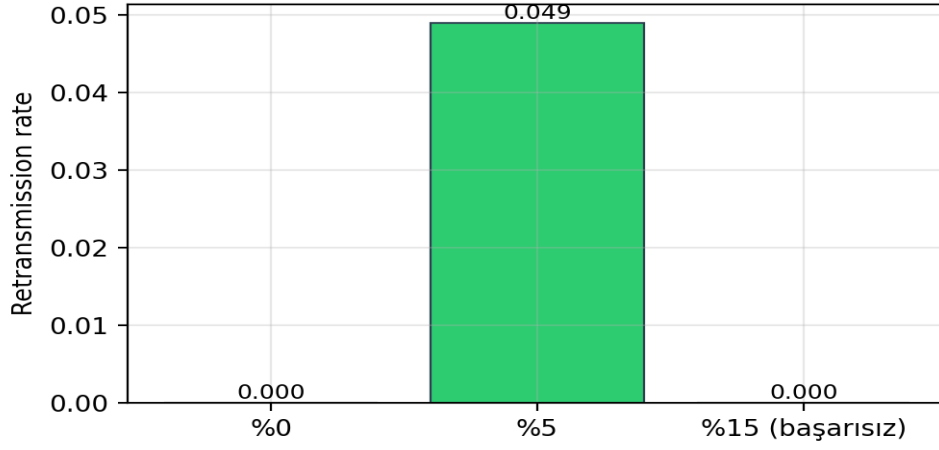
8.3 Senaryo C — Kayıp Oranı (1 MB, simülasyon)

Senaryo C: Kayıp Oranı — Tamamlanma Süresi (1 MB)



Şekil 5. Kayıp oranının tamamlanma süresine etkisi

Senaryo C: Kayıp Oranı — Yeniden Gönderim Oranı



Şekil 6. Kayıp oranının yeniden gönderim oranına etkisi

Yorum: %5 kayıpta ortalama retransmission rate $\approx 4.90\%$, tamamlanma süresi ≈ 46.4 s. Kayıp arttıkça timeout ve retransmit artar; goodput düşer. %15 kayıpta MAX_RETRIES=5 sınırı nedeniyle aktarım başarısız olabilir — bu, protokolün sınırlarını açıkça gösteren beklenen bir sonuçtur.

9. Sonuç ve Değerlendirme

NetProbe, dönem projesi f y nde belirtilen UDP g venilir aktarım, trafik izleme ve performans analizi gereksinimlerini karřılayan  alıřır bir sistemdir. Temel istemci-sunucu mimarisi, ACK/timeout/retransmit mekanizmaları, loglama ve deneysel analiz altyapısı bařarıyla tamamlanmıřtır.

Bonus kapsamında sliding window, kayıp sim lat r  ve HTML dashboard eklenmiřtir. Gelecek  alıřmalarda TCP ile karřılařtırmalı deney ve Wireshark analizi planlanabilir.

Kaynaklar ve Ekler

- Proje kaynak kodu: `netprobe/src/`
- Protokol spesifikasyonu: `docs/protocol.md`
- Deney konfig rasyonları: `experiments/configs/`
- README: Kurulum ve  alıřtırma adımları